The following examples demonstrate some ways to combine indexes, paths, operations, and functions in just about any combination you're likely to need…

Table 11-18. *XPath expression examples*

`[*]`	Has children (is not a leaf node)
`[1]`	First node
`[last()]`	Last node
`[not(last())]`	All nodes except the last
`[position()>1]`	All nodes except the first
`[position()=2]` `[2]`	Second node
`[@class="foo"][1]`	First node with *only* a `foo` class
`[contains(@class, "foo")]` `[1]`	First node that has class `foo` (and any number of others)
`[.//div[@id="foo"]]`	Return node only if it contains `<div id='foo'>` descendant
`ul[count(li) > 2]`	Return `ul` elements with two or more `li` children
`ul[count(li[contains` `(@class, "hide")]) > 0]`	Return `ul` elements containing at least one `<li class="hide">`
`[contains(@class, "foo")` `and position() = 2]`	Return nodes in the second position that also have a `foo` class

… and yep, there's wacky-looking stuff in there. Let's take a minute to look at some of these at work on a real DOM, for which we'll build a more complex XML document, build a `Dom\XMLDocument` object, and throw it at `Dom\XPath` (again, XML is standing in for HTML here, but all the concepts transfer, and this is a deliberate choice on my part… you'll see why):

```
$xml = <<<EOS
<?xml version="1.0" encoding="UTF-8"?>
<data>
  <projects>
    <project class="software javascript">
      <codebase name="frontend" status="live">
        React with 10k dependencies
```